Step Witness

개요

Step Witness는 EVM 실행의 개별 단계(step)를 나타내는 구조체입니다. `ExecStep` 구조체는 각 opcode 실행 또는 특수 상태(BeginTx, EndTx 등)에서 발생하는 모든 정보를 담고 있으며, EVM Circuit에서 실행 흐름을 검증하는 데 사용됩니다.

ExecStep 구조체

정의

`ExecStep` 구조체는 `zkevm-circuits/src/witness/step.rs` 에 정의되어 있으며, 다음과 같은 필드를 포함합니다:

```
pub struct ExecStep {
/// 트랜잭션 호출에서의 인덱스
pub call_index: usize,
/// 이 단계에서 발생한 RW 트레이스의 인덱스들
pub rw_indices: Vec<(RwTableTag, usize)>,
/// 이 단계에서 복사 이벤트를 통해 수행된 RW 연산 수
pub copy_rw_counter_delta: u64,
/// 단계의 실행 상태
pub execution_state: ExecutionState,
/// 단계 이전의 Read/Write 카운터
```

```rust
pub rw_counter: usize,
/// 프로그램 카운터
pub program_counter: u64,
/// 스택 포인터
pub stack_pointer: usize,
/// 남은 가스량
pub gas_left: u64,
/// 이 단계의 가스 비용
pub gas_cost: u64,
/// 메모리 크기 (바이트 단위)
pub memory_size: u64,
/// 단계 시작 시 reversible write 카운터
pub reversible_write_counter: usize,
/// 이 단계에서 발생한 reversible write 수
pub reversible_write_counter_delta: usize,
/// 트랜잭션 내 로그 인덱스 카운터
pub log_id: usize,
/// 단계에 해당하는 opcode
pub opcode: Option<OpcodeId>,
/// 이 단계가 존재하는 블록 번호
pub block_num: u64,
/// 프리컴파일용 보조 데이터
pub aux_data: Option<PrecompileAuxData>,
```

```
}
```

주요 필드 설명

1. 호출 컨텍스트 (call_index)

```
pub call_index: usize
```

- 현재 실행 중인 호출의 인덱스
- 트랜잭션은 여러 호출(CALL, DELEGATECALL 등)을 포함할 수 있음
- 각 호출은 고유한 call_index를 가짐

2. RW 연산 추적 (rw_indices, copy_rw_counter_delta)

```
pub rw_indices: Vec<(RwTableTag, usize)>
pub copy_rw_counter_delta: u64
```

- **rw_indices**: 이 단계에서 발생한 모든 RW 연산의 (태그, 인덱스) 쌍
- **copy_rw_counter_delta**: 복사 이벤트를 통한 추가 RW 연산 수
- RW 연산 타입: Stack, Memory, Storage, Account 등

3. 실행 상태 (execution_state)

```
pub execution_state: ExecutionState
```

- 현재 단계의 실행 상태를 나타내는 열거형
- Opcode 실행 상태 (ADD_SUB, MUL_DIV_MOD, CALL_OP 등)

- 특수 상태 (BeginTx, EndTx, EndBlock, Padding 등)
- 에러 상태 (ErrorInvalidOpcode, ErrorStack, ErrorOutOfGas 등)

4. EVM 상태 (program_counter, stack_pointer, gas_left, memory_size)

```
pub program_counter: u64
pub stack_pointer: usize
pub gas_left: u64
pub memory_size: u64
```

- **program_counter**: 현재 실행 중인 바이트코드의 위치
- **stack_pointer**: 스택의 현재 위치 (STACK_CAPACITY - stack_size)
- **gas_left**: 실행 전 남은 가스
- **memory_size**: 실행 전 메모리 크기 (바이트 단위)

5. 가스 비용 (gas_cost)

```
pub gas_cost: u64
```

- 이 단계를 실행하는 데 소비된 가스
- Opcode별로 다른 가스 비용 적용

6. Reversible Write 추적 (reversible_write_counter, reversible_write_counter_delta)

```
pub reversible_write_counter: usize
pub reversible_write_counter_delta: usize
```

- **reversible_write_counter**: 단계 시작 시 reversible write 카운터
- **reversible_write_counter_delta**: 이 단계에서 발생한 reversible write 수
- 트랜잭션이 revert될 경우 되돌려야 하는 쓰기 연산 추적

7. Opcode (opcode)

```
pub opcode: Option<OpcodeId>
```

- 실행된 opcode (일반 opcode 실행 시)
- BeginTx, EndTx 등 특수 상태에서는 None

8. 프리컴파일 데이터 (aux_data)

```
pub aux_data: Option<PrecompileAuxData>
```

- 프리컴파일 호출 시 필요한 보조 데이터
- Ecrecover, SHA256, MODEXP 등의 프리컴파일에서 사용

실행 단계 데이터 생성

step_convert 함수

`step_convert` 함수는 Bus-Mapping의 실행 단계를 witness 형식으로 변환합니다:

```
pub(super) fn step_convert(
step: &circuit_input_builder::ExecStep,
block_num: u64
```

```
) -> ExecStep
```

변환 과정

1. **RW 인덱스 변환**

```
rw_indices: step

.bus_mapping_instance

.iter()

.map(|x| {

let  tag = match  x.target() {

operation::Target::Memory => RwTableTag::Memory,

operation::Target::Stack => RwTableTag::Stack,

operation::Target::Storage => RwTableTag::AccountStorage,

operation::Target::TransientStorage => RwTableTag::AccountTransientStorage,

operation::Target::TxAccessListAccount => RwTableTag::TxAccessListAccount,

operation::Target::TxAccessListAccountStorage => {

RwTableTag::TxAccessListAccountStorage

}

operation::Target::TxRefund => RwTableTag::TxRefund,

operation::Target::Account => RwTableTag::Account,

operation::Target::CallContext => RwTableTag::CallContext,

operation::Target::TxReceipt => RwTableTag::TxReceipt,

operation::Target::TxLog => RwTableTag::TxLog,

operation::Target::Start => RwTableTag::Start,
```

```
};
(tag, x.as_usize())
})
.collect(),
```

- Bus-Mapping의 연산 타겟을 RwTableTag로 변환
- 각 연산의 인덱스와 함께 저장

2. **실행 상태 변환**

```
execution_state: ExecutionState::from(step),
```

- Bus-Mapping의 ExecState를 witness의 ExecutionState로 변환

3. **스택 포인터 계산**

```
stack_pointer: STACK_CAPACITY - step.stack_size,
```

- EVM 스택은 최대 1024개 요소 (STACK_CAPACITY)
- 스택 포인터는 빈 슬롯의 시작 위치

4. **기타 필드 복사**

```
ExecStep {
call_index: step.call_index,
copy_rw_counter_delta: step.copy_rw_counter_delta,
rw_counter: usize::from(step.rwc),
program_counter: usize::from(step.pc) as u64,
```

```
gas_left: step.gas_left.0,

gas_cost: step.gas_cost.as_u64(),

memory_size: step.memory_size as u64,

reversible_write_counter: step.reversible_write_counter,

reversible_write_counter_delta: step.reversible_write_counter_delta,

log_id: step.log_id,

block_num,

aux_data: step.aux_data.clone(),

// ...

}
```

ExecutionState 변환

From<&ExecError> for ExecutionState

에러 상태를 ExecutionState로 변환:

```
impl From<&ExecError> for ExecutionState {

fn  from(error: &ExecError) -> Self {

match  error {

ExecError::InvalidOpcode => ExecutionState::ErrorInvalidOpcode,

ExecError::StackOverflow | ExecError::StackUnderflow => ExecutionState::ErrorSta

ExecError::WriteProtection => ExecutionState::ErrorWriteProtection,

ExecError::Depth(depth_err) => match  depth_err {
```

```
DepthError::Call => ExecutionState::CALL_OP,

DepthError::Create => ExecutionState::CREATE,

DepthError::Create2 => ExecutionState::CREATE2,

},

ExecError::InsufficientBalance(insuff_balance_err) => match  insuff_balance_err

InsufficientBalanceError::Call => ExecutionState::CALL_OP,

InsufficientBalanceError::Create => ExecutionState::CREATE,

InsufficientBalanceError::Create2 => ExecutionState::CREATE2,

},

// ... 기타 에러 타입

ExecError::OutOfGas(oog_error) => match  oog_error {

OogError::Constant => ExecutionState::ErrorOutOfGasConstant,

OogError::StaticMemoryExpansion => {

ExecutionState::ErrorOutOfGasStaticMemoryExpansion

}

OogError::DynamicMemoryExpansion => {

ExecutionState::ErrorOutOfGasDynamicMemoryExpansion

}

// ... 기타 OOG 에러

},

}

}
```

```
}
```

From<&circuit_input_builder::ExecStep> for ExecutionState

정상 실행 상태를 ExecutionState로 변환:

```
impl From<&circuit_input_builder::ExecStep> for ExecutionState {
fn from(step: &circuit_input_builder::ExecStep) -> Self {
if let Some(error) = step.error.as_ref() {
return error.into();
}
match step.exec_state {
circuit_input_builder::ExecState::Op(op) => {
// 특수 opcode 그룹 처리
if op.is_dup() {
return ExecutionState::DUP;
}
if op.is_push() {
return ExecutionState::PUSH;
}
if op.is_swap() {
return ExecutionState::SWAP;
}
if op.is_log() {
```

```
return ExecutionState::LOG;

}

// 개별 opcode 매핑

match  op {

OpcodeId::ADD | OpcodeId::SUB => ExecutionState::ADD_SUB,

OpcodeId::MUL | OpcodeId::DIV | OpcodeId::MOD => ExecutionState::MUL_DIV_MOD,

OpcodeId::ADDMOD => ExecutionState::ADDMOD,

OpcodeId::MULMOD => ExecutionState::MULMOD,

OpcodeId::EQ | OpcodeId::LT | OpcodeId::GT => ExecutionState::CMP,

OpcodeId::MLOAD | OpcodeId::MSTORE | OpcodeId::MSTORE8 => ExecutionState::MEMORY

OpcodeId::SLOAD => ExecutionState::SLOAD,

OpcodeId::SSTORE => ExecutionState::SSTORE,

OpcodeId::CALL | OpcodeId::CALLCODE | OpcodeId::DELEGATECALL | OpcodeId::STATICC

ExecutionState::CALL_OP

}

OpcodeId::CREATE => ExecutionState::CREATE,

OpcodeId::CREATE2 => ExecutionState::CREATE2,

// ... 기타 opcode

_ => unimplemented!("unimplemented opcode {:?}", op),

}

}

circuit_input_builder::ExecState::Precompile(precompile) => match  precompile {

PrecompileCalls::Ecrecover => ExecutionState::PrecompileEcrecover,
```

```
PrecompileCalls::Sha256 => ExecutionState::PrecompileSha256,
PrecompileCalls::Modexp => ExecutionState::PrecompileBigModExp,
PrecompileCalls::Bn128Add => ExecutionState::PrecompileBn256Add,
PrecompileCalls::Bn128Mul => ExecutionState::PrecompileBn256ScalarMul,
PrecompileCalls::Bn128Pairing => ExecutionState::PrecompileBn256Pairing,
// ... 기타 프리컴파일
},
circuit_input_builder::ExecState::BeginTx => ExecutionState::BeginTx,
circuit_input_builder::ExecState::EndTx => ExecutionState::EndTx,
circuit_input_builder::ExecState::EndBlock => ExecutionState::EndBlock,
circuit_input_builder::ExecState::Padding => ExecutionState::Padding,
}
}
}
```

ExecStep의 유틸리티 메서드

memory_word_size 메서드

```rust
pub fn memory_word_size(&self) -> u64 {
// EVM은 항상 메모리 크기를 워드 크기로 패딩
assert_eq!(self.memory_size % N_BYTES_WORD as u64, 0);
self.memory_size / N_BYTES_WORD as u64
```

```
}
```

- 이 단계 **이전**의 메모리 크기를 워드 단위로 반환
- EVM은 메모리를 32바이트(워드) 단위로 관리
- 메모리 크기는 항상 32의 배수

ExecutionState 열거형

주요 실행 상태

1. Opcode 실행 상태

```
// 산술 연산
ADD_SUB, MUL_DIV_MOD, ADDMOD, MULMOD, SDIV_SMOD, EXP

// 비교 연산
CMP, SCMP, ISZERO

// 비트 연산
BITWISE, NOT, BYTE, SHL_SHR, SAR, SIGNEXTEND

// 스택 연산
PUSH, POP, DUP, SWAP

// 메모리 연산
```

MEMORY, MSIZE, MCOPY

// 스토리지 연산

SLOAD, SSTORE, TLOAD, TSTORE

// 제어 흐름

JUMP, JUMPI, JUMPDEST, PC, GAS

// 호출 관련

CALL_OP, CREATE, CREATE2, RETURN_REVERT

// 블록 컨텍스트

BLOCKCTXU64, BLOCKCTXU160, BLOCKCTXU256, BLOCKHASH

// 트랜잭션 컨텍스트

CALLER, CALLVALUE, CALLDATALOAD, CALLDATASIZE, CALLDATACOPY

CODECOPY, CODESIZE, EXTCODECOPY, EXTCODESIZE, EXTCODEHASH

RETURNDATASIZE, RETURNDATACOPY

// 로그

LOG

// 해시

```
SHA3


// 기타

SELFBALANCE, BALANCE, ADDRESS, ORIGIN, GASPRICE, CHAINID
```

2. 특수 상태

```
BeginTx // 트랜잭션 시작

EndTx // 트랜잭션 종료

EndBlock // 블록 종료

EndInnerBlock // 내부 블록 종료 (멀티 블록)

Padding // 패딩
```

3. 프리컴파일 상태

```
PrecompileEcrecover // 0x01: ECDSA 복구

PrecompileSha256 // 0x02: SHA-256

PrecompileRipemd160 // 0x03: RIPEMD-160

PrecompileIdentity // 0x04: Identity

PrecompileBigModExp // 0x05: 모듈러 지수 연산

PrecompileBn256Add // 0x06: BN256 점 덧셈

PrecompileBn256ScalarMul // 0x07: BN256 스칼라 곱셈

PrecompileBn256Pairing // 0x08: BN256 페어링
```

```
PrecompileBlake2f // 0x09: Blake2f
```

4. 에러 상태

```
// 일반 에러
ErrorInvalidOpcode
ErrorStack // 스택 오버플로우/언더플로우
ErrorWriteProtection
ErrorInvalidJump
ErrorInvalidCreationCode
ErrorReturnDataOutOfBound
ErrorCodeStore
ErrorPrecompileFailed


// Out of Gas 에러
ErrorOutOfGasConstant
ErrorOutOfGasStaticMemoryExpansion
ErrorOutOfGasDynamicMemoryExpansion
ErrorOutOfGasMemoryCopy
ErrorOutOfGasAccountAccess
ErrorOutOfGasLOG
ErrorOutOfGasEXP
ErrorOutOfGasSHA3
ErrorOutOfGasCall
```

```
ErrorOutOfGasPrecompile

ErrorOutOfGasSloadSstore

ErrorOutOfGasCREATE

ErrorOutOfGasSELFDESTRUCT
```

ExecStep의 사용

EVM Circuit에서의 사용

ExecStep은 EVM Circuit에서 다음과 같이 사용됩니다:

1. **Step 구조 생성**

```
let step = Step::new(
&self.execution,
&mut self.step_curr,
call,
&mut self.cell_manager,
);
```

2. **ExecutionGadget 선택**

```
let execution_gadget = match step.execution_state {
ExecutionState::ADD_SUB => Box::new(AddSubGadget::construct(...)),
ExecutionState::MUL_DIV_MOD => Box::new(MulDivModGadget::construct(...)),
// ... 기타 상태
```

```
};
```

3. **제약 조건 적용**

```
execution_gadget.assign_exec_step(
region,
offset,
block,
transaction,
call,
step,
)?;
```

RW 연산 접근

```
for (rw_tag, rw_idx) in &step.rw_indices {
let  rw = &block.rws[*rw_idx];
// RW 연산 처리
}
```

요약

Step Witness는 EVM 실행의 개별 단계를 상세히 기록합니다:

1. **실행 컨텍스트**: call_index, program_counter, stack_pointer 등

2. **상태 변화**: RW 연산, 가스 소비, 메모리 확장 등

3. **실행 상태**: Opcode, 프리컴파일, 특수 상태, 에러 상태

4. **추적 정보**: reversible write, 로그 ID, 복사 카운터 등

5. **변환 로직**: Bus-Mapping의 ExecStep을 witness 형식으로 변환

ExecStep은 `step_convert` 함수를 통해 생성되며, EVM Circuit에서 각 실행 단계의 정확성을 검증하는 데 사용됩니다. ExecutionState는 각 단계의 타입을 나타내며, 해당하는 ExecutionGadget이 적절한 제약 조건을 적용합니다.